# Neural-Enhanced Topology-Aware Graph Indexing for High-Dimensional Approximate Nearest Neighbor Search

Anonymous Author(s)

## Abstract

Approximate Nearest Neighbor Search (ANNS) is a key technique for high-dimensional vector retrieval, with graph-based indexing as its mainstream implementation, yet existing structures have clear limitations in practice. Hierarchical organizations often incur substantial *runtime memory* overhead, while still lacking global topological awareness in routing. In contrast, standard proximity graphs typically adopt a fixed, *query-agnostic* routing topology limited by over-reliance on local Euclidean distance, and their performance may become unstable under shifts in the query distribution.

To address these issues of topological-geometric misalignment, we propose **NETG**, a novel framework featuring an Asymmetric Latent Routing mechanism to bridge the topological and geometric gaps in graph search. NETG strictly decouples offline topological reasoning from online serving. In the offline stage, it mines topological hubs and employs a neural network to "fold" their coordinates in a latent metric space, effectively drawing topologically connected but geometrically distant regions together. In the online stage, queries remain strictly in their original feature space without incurring any neural inference overhead (0ms). By constructing a micro-scale navigation graph exclusively on these offline-folded hubs, NETG allows the search agent to precisely "teleport" to the optimal target basin before executing the fine-grained base graph search. Experiments on large-scale benchmark datasets show that NETG achieves higher throughput in high-recall regimes, improving QPS by **up to 11.3%** under strict isorecall conditions (i.e., conditions where both algorithms achieve the exact same recall) over state-of-the-art baselines while introducing negligible additional memory overhead.

## CCS Concepts

• **Information systems** → Nearest-neighbor search; Retrieval efficiency; Indexing methods; • **Computing methodologies** → Neural networks.

## Keywords

Approximate Nearest Neighbor Search; Graph Indexing; Neural Manifold Folding; Topology Awareness; Ant Colony Optimization

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

*Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Jeju, Republic of Korea*

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/26/08
https://doi.org/XXXXXXX.XXXXXXX

**ACM Reference Format:**
Anonymous Author(s). 2026. Neural-Enhanced Topology-Aware Graph Indexing for High-Dimensional Approximate Nearest Neighbor Search. In *Proceedings of Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, New York, NY, USA, 11 pages. https://doi.org/XXXXXXX.XXXXXXX

## 1 Introduction

High-dimensional vector retrieval is the core foundation of modern unstructured data processing, where Approximate Nearest Neighbor Search (ANNS) is the key enabling technique[cite: 238]. Among all mainstream ANNS paradigms, graph-based approaches have become the backbone of modern retrieval systems [3, 40, 50, 53][cite: 239]. The fundamental philosophy of graph indexing is to transform the exhaustive distance comparisons in the continuous metric space into an efficient discrete routing problem on topological structures. By navigating along the edges of a proximity graph, the search agent approximates the nearest neighbors through a sequence of local greedy decisions.

While current methodologies have extensively investigated designing efficient index structures to drive search execution, the initialization of this process—specifically, where the search begins—remains a critical yet underexplored bottleneck. Among existing approaches, NSG (Navigating Spreading Graph) [8, 18, 19, 25] represents the state-of-the-art in monotonic routing efficiency (i.e., the property where each routing step strictly decreases the distance to the query)[cite: 240]. However, despite their success on simple distributions, standard graph indices suffer from a fundamental limitation we term "Geometric Myopia." [cite: 241] **Because their entry point selection (e.g., relying on a static geometric medoid) is highly dependent on local Euclidean distances, it frequently falls into high-dimensional geometric traps before the routing even effectively begins.**

They rely heavily on local Euclidean distances for greedy routing and entry point selection, implicitly assuming isotropic data manifolds. Consequently, their entry selection frequently falls into high-dimensional geometric traps before the routing even effectively begins. Furthermore, deep embeddings often exhibit nonlinear structures and extreme skewness [4, 17, 42]. While hierarchical graphs like HNSW [35] attempt to mitigate local optima by constructing multi-layer shortcuts, they solve this in a rather cumbersome "Space-for-Time" strategy. To ensure navigability, HNSW maintains redundant edges across upper layers, causing significant memory bloat [5, 21]. Moreover, their randomized entry selection remains geometrically myopic, often failing to navigate complex voids in high-dimensional manifolds [38], leaving substantial room for algorithmic optimization in search initialization.

In such complex topologies, a "Geometric Void"—where greedy routing gets trapped in local optima—frequently severs the path

to the true nearest neighbor [6, 33, 48, 54]. We argue that the routing bottleneck lies in the misalignment between static geometric metrics and topological structures.

To bridge this efficiency gap, we propose **NETG** (Neural-Enhanced Topology-aware Graph), an effective approach for initializing graph-based search that pivots from the traditional "Space-for-Time" trade-off to an "Intelligence-for-Space" paradigm. Unlike methods that demand heavy runtime restructuring, NETG operates as a strictly **decoupled offline-to-online architecture**.

Our approach orchestrates two asymmetric phases. In the **Offline Distillation Phase**, we engineer a **Supervision Generation Pipeline**. By combining swarm intelligence (ACO) [9, 12] and density-based clustering (W-DPC) [41, 52, 59], we rigorously mine the "Manifold Backbone" to identify robust entry hubs. These components act as a temporary *computational scaffold*—generating ground-truth navigation signals—and are **discarded** immediately after training to ensure zero runtime bloat.

In the **Online Inference Phase**, the system operates with strictly **zero online neural inference overhead (0ms)**. Instead of relying on a runtime surrogate, we propose an **Asymmetric Latent Routing** mechanism [26, 47]. The neural network is exclusively used offline to "fold" the coordinates of the mined hubs. During online search, queries remain in their raw feature space and navigate a Micro-HNSW graph built purely on these folded hubs. This allows the search agent to precisely "teleport" across geometric voids to the optimal target basin before executing the fine-grained base graph search.

Extensive experiments on benchmarks like Deep10M and SIFT10M-Skewed [2, 4, 44] show that NETG yields up to **11.3% higher QPS** than the standard NSG baseline while maintaining a minimal memory footprint (< 0.01% overhead).

In summary, our contributions are:

- **Offline Topological Distillation**: A strictly offline-online decoupled pipeline that extracts the intrinsic navigation backbone of high-dimensional data via ACO-based modeling [9, 11], with zero residual overhead in online serving.
- **Robust Topology-Aware Entry Selection**: A preference-based W-DPC algorithm [41] that prioritizes high-throughput navigation nodes over geometric centroids, with strong robustness to data skewness.
- **Asymmetric Zero-Overhead Routing**: An asymmetric neural routing mechanism realizing the "Intelligence-for-Space" paradigm, achieving absolute zero online neural inference latency [51, 56] via offline-only hub optimization and lightweight Micro-HNSW navigation.
- **Taxonomy-Driven Auto-Configuration**: A geometric/semantic taxonomy-based auto-configuration scheme that eliminates manual hyperparameter tuning, lowering the engineering barrier for industrial deployment.

# 2 Preliminaries & Motivation

## 2.1 Problem Formulation

Consider a finite dataset $\mathcal{X} = \{x_1, \ldots, x_N\} \subset \mathbb{R}^D$ residing in a $D$-dimensional Euclidean space. Given a query vector $q \in \mathbb{R}^D$, the goal of Nearest Neighbor Search (NNS) is to identify $x^* = \arg\min_{x \in \mathcal{X}} \delta(q, x)$, where $\delta(\cdot, \cdot)$ denotes the Euclidean distance.

Since exact NNS is computationally prohibitive at scale, we focus on Approximate NNS (ANNS), which aims to find an $x'$ such that $\delta(q, x') \leq (1 + \epsilon)\delta(q, x^*)$ with high probability and low latency.

**Graph-based Indexing.** Methods like NSG [19, 39] and HNSW [1, 35] construct a base graph $G = (V, E)$, where vertices $V$ map to data points in $\mathcal{X}$. The search process is typically modeled as *Greedy Routing*. Starting from an entry point $v_0$ (e.g., a graph medoid or a hierarchical layer entrance), the search agent iteratively moves to a neighbor closer to $q$:

$$v_{t+1} = \arg\min_{u \in \mathcal{N}(v_t)} \delta(q, u), \quad \text{s.t.} \quad \delta(q, u) < \delta(q, v_t) \tag{1}$$

where $\mathcal{N}(v_t)$ is the neighbor set of $v_t$. The routing terminates at a node $v_T$ when no neighbor is closer to $q$, i.e., $\forall u \in \mathcal{N}(v_T), \delta(q, u) \geq \delta(q, v_T)$.

Ideally, $v_T$ should be the global nearest neighbor $x^*$. However, in practice, $v_T$ is often merely a *local optimum*, especially when the graph topology fails to align with the query's proximity field.

**The Entry Point Bottleneck.** This vulnerability highlights a critical dimension of graph-based ANNS: the absolute dependency on the initial entry point $v_0$. The success rate and efficiency of greedy routing are intrinsically bound by the topological viability of $v_0$. If $v_0$ resides on a disconnected manifold branch relative to $x^*$, the strict monotonic constraint forces the search agent into a topological dead-end, regardless of the base graph's density.

Therefore, the core problem addressed in this work extends beyond static graph construction. We formally redefine the routing challenge as learning an optimal entry mapping function $\mathcal{F} : \mathbb{R}^D \to \mathcal{H} \subset V$, where $\mathcal{H}$ is a designated set of topological hubs. For any given query $q$, the objective is to yield a starting node $v_0 = \mathcal{F}(q)$ that minimizes the navigation cost (routing hops) while maximizing the convergence probability $\mathbb{P}(v_T = x^*)$, thereby explicitly resolving the misalignment between local geometric heuristics and global manifold topology.

## 2.2 The Pitfall of Geometric Greedy Routing

The difficulty in finding the optimal $v_0$ primarily stems from the inherent bias of traditional routing strategies. Greedy routing operates on the premise that the local gradient of Euclidean distance serves as a reliable proxy for global proximity. While this heuristic holds for convex or uniform distributions, it breaks down in the non-linear, complex manifolds typical of modern deep embeddings [4, 57].

**Metric Misalignment.** Consider the "Nested Tori" manifold depicted in Figure 1, where the data surface convolutes. Points $A$ and $B$, situated on adjacent layers, exhibit high spatial proximity in $\mathbb{R}^D$ (small Euclidean distance) despite being distant along the manifold's geodesic surface. Standard graph construction algorithms, constrained by their reliance on raw $\delta$, typically forge edges based on this misleading geometric proximity, failing to encode the high-level connectivity necessary to circumnavigate the inter-layer voids.

**The "Geometric Void" Trap.** When a query $q$ lands near such a structural gap, greedy routing becomes treacherous. The search agent, algorithmically bound to minimize Euclidean distance, is lured towards the gap, only to encounter a lack of valid bridging edges. We term this phenomenon a **Geometric Void**—a topological

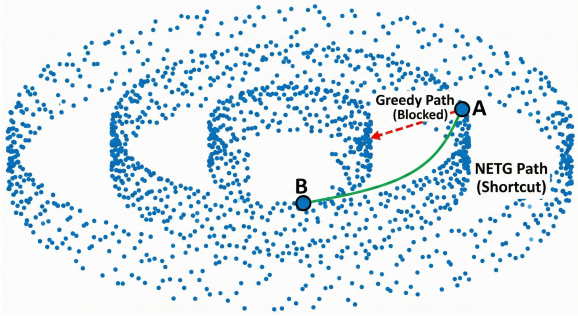



**Figure 1: Illustration of Geometric Myopia. In a non-linear manifold (e.g., Nested Tori), nodes $A$ and $B$ are spatially adjacent ($\delta(A, B) \approx 0$) but reside on disjoint topological components. Standard greedy routing (red dashed) often stalls at the *Geometric Void* due to the lack of inter-layer connectivity. While HNSW's hierarchical shortcuts (blue dotted) can alleviate local minima, their heuristic, query-agnostic construction may yield suboptimal jumps on highly non-linear manifolds when applied to the raw space, incurring extra exploration. Conversely, NETG (green solid) perceives the latent global topology, executing a precise jump to the target basin via neural guidance.**

dead-end where the search process prematurely converges to a local optimum, stranding the agent far from the true neighbor.

**Inefficacy of Probabilistic Hierarchies in Raw Space.** Hierarchical structures like HNSW [35] attempt to alleviate geometric myopia by introducing multi-scale shortcuts. However, when applied directly to the raw metric space of millions of nodes, these heuristics still depend on the deceptive Euclidean metric. On non-convex manifolds, a geometrically long edge is liable to inadvertently "tunnel" through a void, depositing agents in disconnected regions. Furthermore, to ensure navigability across massive graphs, HNSW must maintain redundant edges across upper layers, causing significant memory bloat.

**Necessity of Asymmetric Latent Routing.** Overcoming these limitations demands an indexing mechanism that decouples topological reasoning from online search. Instead of building a massive hierarchy, the system must learn to "fold" the manifold in a latent metric space exclusively for entry points. We propose an **Asymmetric Latent Space**: during the offline phase, we use neural networks to distort the coordinates of a tiny set of topological Hubs, effectively bringing topologically connected but geometrically distant regions together. During the online phase, queries remain strictly in their original raw feature space, resulting in zero online neural inference overhead. This allows us to build a Micro-HNSW graph exclusively on the few hundreds of folded Hubs—eliminating the memory bloat of traditional HNSW while exploiting its extreme speed for perfect entry point routing.

# 3 Methodology

## 3.1 System Architecture

Instead of a monolithic index, NETG is architected as two decoupled computational streams (see Figure **??**):

- **Stream 1: Topology Mining and Manifold Folding (Offline).** We frame index construction as a topology mining task. Given a base graph $G$, the pipeline deploys **Exploration Agents** (via ACO) to probe the implicit backbone and utilizes W-DPC to lock onto high-value hubs. It then trains a neural network to "fold" these hubs in a latent metric space, exporting their distorted coordinates as a compact topological representation. The computationally heavy training process is decommissioned post-training.
- **Stream 2: Zero-Overhead Asymmetric Latent Routing (Online).** The inference engine is stripped to its bare essentials: the raw graph topology and a Micro-HNSW graph built purely on the offline-folded hubs. For an incoming query $q$ (which remains in its raw feature space), the system executes a rapid routing on the micro-graph to pinpoint the optimal "hot-start" entry position. This is then seamlessly handed over to the standard greedy routing on the base graph with strictly **zero online neural inference overhead (0ms)**.

**Lightweight Integration.** A defining feature of this architecture is its minimal footprint and absolute zero inference latency. Since the heavy lifting (neural projection) is offloaded entirely to the offline stream, the online component requires negligible extra memory (storing only the folded hub coordinates, typically $< 0.1\%$ of the graph size) and eliminates the need for runtime tensor operations. This makes NETG a highly efficient, "Plug-and-Play" enhancer for existing graph indices [3, 13, 50].

## 3.2 Manifold Backbone Mining via ACO

Standard graph indices like NSG typically treat all nodes equally or rely on static geometric centrality (e.g., the graph Medoid) to initiate search. However, as discussed in Section 2.2, in skewed high-dimensional manifolds, the geometric center is often topologically isolated from the majority of data clusters [4]. To capture the intrinsic *navigational structure* of the data, we propose to mine the **Manifold Backbone** using a modified Ant Colony Optimization (ACO) [9, 11].

*3.2.1 Graph Traversal as a Reinforcement Process.* We model the graph traversal on the base graph $G = (V, E)$ as a Markov Decision Process (MDP). A swarm of agents (simulated ants) is deployed to explore the graph [14, 23]. Unlike traditional shortest-path algorithms (e.g., Dijkstra), our agents are designed to learn a probabilistic routing policy that dynamically balances geometric greediness with topological exploration.

Let $\tau_{ij}(t)$ denote the *pheromone* level on edge $e_{ij}$ at iteration $t$, representing the accumulated topological experience (ie., the historical "popularity" of this transition in successful searches). Let $\eta_{ij}$ be the heuristic visibility, prioritizing geometrically closer neighbors:

$$\eta_{ij} = \frac{1}{\delta(v_i, v_j) + \epsilon} \tag{2}$$

where $\delta(\cdot, \cdot)$ is the Euclidean distance and $\epsilon$ is a small constant for numerical stability.

*3.2.2 Probabilistic Transition Rule.* During the offline training phase, an ant agent located at node $v_i$ selects the next neighbor $v_j \in \mathcal{N}(v_i)$

stochastically. The probability $P(v_j|v_i)$ follows a softmax-like distribution weighted by hyperparameters $\alpha$ and $\beta$:

$$P(v_j|v_i) = \frac{[\tau_{ij}(t)]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{v_k \in \mathcal{N}(v_i)} [\tau_{ik}(t)]^\alpha \cdot [\eta_{ik}]^\beta} \tag{3}$$

Here, $\alpha$ controls the influence of historical topological knowledge, while $\beta$ controls the influence of local geometric gradients. To implement this efficiently, we employ Roulette Wheel Selection [9, 12], ensuring that nodes with higher combined probability mass are more likely to be chosen, while still allowing for occasional exploration of lower-probability paths.

*3.2.3 Multi-Objective Pheromone Update.* The learning process proceeds in $T$ iterations. In each iteration, a batch of $M$ ants is instantiated with random start nodes to simulate the actual retrieval workload. **To provide clear navigation targets, each ant $k$ is assigned a specific target query $q$ sampled from a training workload. The resulting search trajectory $\mathcal{P}_k = \{v_1, v_2, \ldots, v_T\}$ represents the sequence of nodes visited by the ant in its attempt to reach $q$.** Once an agent $k$ completes the trajectory, we perform a global pheromone update.

A critical innovation in NETG is the design of a Multi-Objective Reward Function. Unlike classic ACO which typically optimizes for a single objective, our update rule enforces a **multiplicative joint reward**. A path must be both Accurate and Efficient to receive a high score; failing in either metric severely penalizes the overall reward. The pheromone update rule is defined as:

$$\tau_{ij}(t+1) = (1-\xi) \cdot \tau_{ij}(t) + \sum_{k=1}^{M} \Delta\tau_{ij}^k \tag{4}$$

where $\xi \in (0, 1)$ is the pheromone evaporation coefficient.

The reward term $\Delta\tau_{ij}^k$ is calculated as:

$$\Delta\tau_{ij}^k = Q \cdot \underbrace{\left(\frac{1}{\text{Error}(\mathcal{P}_k) + \epsilon}\right)}_{\text{Accuracy Score}} \cdot \underbrace{\left(\frac{1}{\log(|\mathcal{P}_k| + 2)}\right)}_{\text{Efficiency Score}} \tag{5}$$

where $Q$ is a scaling constant (e.g., 100), and $\text{Error}(\mathcal{P}_k)$ represents the rank error of the terminal node $v_T$ reached by path $\mathcal{P}_k$. Formally, it is defined as the number of data points in the dataset $\mathcal{X}$ that are geometrically closer to the query $q$ than $v_T$:

$$\text{Error}(\mathcal{P}_k) = \sum_{x \in \mathcal{X}} \mathbb{I}\big(\delta(q, x) < \delta(q, v_T)\big) \tag{6}$$

where $Q$ is a scaling constant (e.g., 100), and $\text{Error}(\mathcal{P}_k)$ represents the relative distance error of the terminal node $v_T$ reached by path $\mathcal{P}_k$. To avoid the prohibitive $O(N)$ computational overhead of evaluating global rank error during offline training, we utilize the pre-computed Ground Truth (GT) to achieve $O(1)$ evaluation. Formally, it is defined as:

$$\text{Error}(\mathcal{P}_k) = \frac{\delta(q, v_T) - \delta(q, x^*)}{\delta(q, x^*) + \epsilon} \tag{7}$$

where $x^*$ is the true nearest neighbor of query $q$, and $\epsilon$ is a smoothing term to prevent division by zero. The **Accuracy Score** ensures that only trajectories leading closely to the ground truth (i.e., $\text{Error}(\mathcal{P}_k) \to 0$) contribute significantly to the backbone. The **Efficiency Score** penalizes long, circuitous detours. The logarithmic penalty $\log(|\mathcal{P}_k|)$ prevents over-penalizing necessary hops in sparse regions while filtering out redundant loops [27, 29].

*3.2.4 Log-Normalization and Backbone Emergence.* After training, the raw pheromone values often exhibit a long-tail distribution, where a few "super-highways" accumulate exponentially higher weights than local roads. To ensure numerical stability for the subsequent clustering stage, we apply a Log-Normalization strategy to the aggregated node pheromones [10, 20, 24]:

$$w_i = \frac{\log(1 + \sum_{v_j \in \mathcal{N}(v_i)} \tau_{ji}) - \min(\log \tau)}{\max(\log \tau) - \min(\log \tau)} \tag{8}$$

This normalization maps the topological importance into a bounded range. Through the competition between evaporation ($\xi$) and reinforcement ($\Delta\tau$), edges that frequently participate in optimal routing paths for diverse queries emerge as the **Manifold Backbone**. This weighted subgraph highlights the critical "bridges" across geometric voids, providing robust candidates for entry point selection in the next stage.

## 3.3 Robust Topological Hub Selection via Hierarchical W-DPC

The efficacy of monotonic search graphs is contingent upon the quality of the initial topological hubs. An initialization that is topologically disconnected from the query target necessitates a lengthy "migration phase," severely degrading query throughput. While partition-based methods like K-Means are standard, they minimize global geometric variance, frequently positioning centroids in "Geometric Voids"—sparse regions lacking data support. This misalignment is particularly acute in deep embeddings, where data resides on non-convex manifolds rather than in spherical clusters [15].

To rectify this, we introduce Weighted Density Peak Clustering (W-DPC). Distinct from the original DPC algorithm [41], we reformulate the density metric using learned topological priors and implement a Two-Layer Hierarchical Strategy to resolve the scale and skewness inherent in high-dimensional data [36, 52].

*3.3.1 Topological Density Formulation.* We discard the geometric neighbor counting used in standard DPC. Instead, the Weighted Density $\rho_i$ of a node $v_i$ is directly instantiated as its log-normalized navigational weight $w_i$ derived from the ACO phase (Eq. 8).

This formulation fundamentally alters the clustering objective: rather than identifying regions of high *spatial concentration*, $\rho_i$ identifies regions of high *navigational throughput*. Consequently, nodes located in dense but topologically dead-ended clusters are naturally suppressed, while critical "bridge" nodes on the manifold backbone are prioritized.

*3.3.2 Scalable Hybrid Delta Approximation.* The canonical DPC algorithm requires computing the dependent distance $\Delta_i = \min_{j:\rho_j > \rho_i} \delta(v_i, v_j)$, entailing a prohibitive $O(N^2)$ complexity. To scale to million-sized datasets, we devise a Hybrid Approximation strategy that exploits the pre-built proximity graph structure [31]:

- **Graph Traversal Lookahead:** For node $v_i$, we iterate through its adjacency list $\mathcal{N}(v_i)$ in the base graph. Given the graph's small-world property, the "parent" node (a neighbor

**Algorithm 1:** Hierarchical W-DPC with DASP

**Input:** Graph $G$, Weights $W$, Threshold $\theta_{giant}$
**Output:** Entry Set $\mathcal{E}$
// Phase 1: Global Coarse Clustering
Assign density $\rho_i \leftarrow W[i]$ for all $v_i \in V$
Compute $\Delta_i$ via Hybrid Approximation
Generate initial clusters $C_{init}$ based on $\psi_i = \rho_i \times \Delta_i$
$\mathcal{E} \leftarrow$ Top peaks from $C_{init}$
// Phase 2: DASP Refinement
**foreach** *Cluster* $C_k \in C_{init}$ **do**
  **if** $|C_k| > \theta_{giant}$ **then**
    // Re-cluster specifically within Giant Cluster
    Extract subgraph $G_k$ induced by $C_k$
    Re-normalize $\rho^{local}$ relative to $C_k$
    Compute intra-cluster $\Delta'$ and $\psi'$
    $e_{sub} \leftarrow \arg\max_{v \in C_k} \psi'$
    $\mathcal{E} \leftarrow \mathcal{E} \cup \{e_{sub}\}$
  **end**
**end**
**return** $\mathcal{E}$

with higher density) is statistically likely to be found within 1-2 hops.

- **Stochastic Sampling:** In the rare event that all local neighbors have lower densities ($\rho_j \leq \rho_i$), we sample a candidate set $S$ from the global high-density population to estimate $\Delta_i$.

This hybrid approach effectively bounds the search space, reducing complexity to linear time relative to the dataset size $N$.

*3.3.3 Hierarchical Refinement via DASP.* A persistent pathology in massive deep embedding datasets is the "Giant Cluster" phenomenon, where a single dominant peak absorbs the majority of data points, masking local substructures [52, 59]. Standard DPC reduces such complex manifolds to a single entry point, causing under-coverage of peripheral regions.

We address this via Density-Adaptive Sub-peak Promotion (DASP), a recursive refinement mechanism implemented as a second clustering layer (Algorithm 1). First, global clusters are formed. Any cluster $C_{giant}$ exceeding a size threshold $\theta_{giant}$ (e.g., $N/50$) triggers the refinement process. Within $C_{giant}$, densities are re-normalized to the local scope, and auxiliary decision scores $\psi'$ are computed to identify Sub-peaks—nodes that are locally dominant despite being globally overshadowed by the primary peak. These sub-peaks are promoted to the entry set $\mathcal{E}$, ensuring uniform spatial coverage across the manifold.

*3.3.4 Complexity Analysis.* Through the aforementioned optimizations, our hierarchical W-DPC algorithm achieves near-linear scalability. The topological density assignment operates in $O(1)$. The Hybrid Delta Approximation runs in $O(N \cdot (K_{graph} + S_{sample}))$, where $K_{graph}$ is the graph degree ($\approx 32$) and $S_{sample}$ is a constant. The DASP refinement only processes the subset of nodes within giant clusters. Empirically, the entire W-DPC procedure consumes less than 5% of the total index construction time on the Deep10M benchmark.

## 3.4 Neural Manifold Folding & Asymmetric Latent Routing

While W-DPC yields a robust set of candidate topological hubs $\mathcal{H}$, the challenge of assigning the *optimal* starting point for a specific query $q$ persists. The conventional heuristic—selecting the geometrically nearest hub $\arg\min_{h \in \mathcal{H}} \delta(q, h)$—suffers from the "Geometric Myopia" delineated in Section 2.2. Within deep embedding spaces, the Euclidean nearest hub is frequently situated on a disconnected manifold branch, separated from the query target by a low-density "void." Initiating the search from such a topological dead-end necessitates a costly U-shaped detour, severely degrading query latency [48, 55].

To bridge this gap, we introduce **Neural Manifold Folding**, a learning-based mechanism that decouples coordinate representation from the raw feature space. Instead of relying on a runtime surrogate, this facilitates **Asymmetric Latent Routing**: the capacity to fold the manifold offline and "teleport" the search agent online, thereby circumventing geometric voids with absolute zero online neural inference overhead.

*3.4.1 Heuristic Supervision Mining.* Calculating the theoretically optimal entry point requires exhaustive graph traversal—a luxury we cannot afford during online inference. Instead, we implement an **Offline Supervision Mining** protocol. We deploy a **Bounded Exploration Agent** to scout for empirically superior entry hubs, thereby generating ground-truth labels for the Neural Projector [32].

**Agent Policy & Label Generation.** The agent mimics actual retrieval behavior. For a given training query $q$, it initiates greedy routing from every candidate hub in $\mathcal{H}$. The hub that yields the target neighbor with the minimal traversal steps (cost) is locked as the **Target Entry Label** ($p^+$). If multiple hubs succeed, preference is given to the one minimizing computational cost. The precise mining logic is detailed in Algorithm 2.

where $\text{Cost}(h \to q)$ denotes the empirical search cost from the hub $h$ to the ground truth of query $q$, strictly measured by the Number of Distance Computations (NDC). The ProbeSearch subroutine acts as an Oracle Simulator that executes a greedy routing on the base graph $G$. To ensure computational feasibility and reflect realistic online search behaviors, this exploration is bounded by a maximum candidate pool size $L$ (beam width). If the routing successfully converges to the ground truth within the bounded pool, it returns the accumulated NDC; otherwise, it assigns an infinite penalty.

**Contrastive Hard Negative Mining.** To equip the model with discriminative power, knowing "where to start" is insufficient. The agent actively identifies **Deceptive Negatives**—hubs that are geometrically close to the query but lead the greedy search into local optima dead-ends. By constructing contrastive triplets between these "Geometric Traps" and the discovered "Topological Shortcuts," the model is forced to look beyond superficial Euclidean proximity [16, 37].

*3.4.2 Asymmetric Neural Projector.* To achieve zero-overhead online routing, we must abandon interaction-heavy architectures (e.g., Cross-Encoders) that require real-time query participation. Instead,

**Algorithm 2:** Offline Navigation Label Mining

**Input:** Graph $G$, Hubs $\mathcal{H}$, Query Set $\mathcal{Q}_{train}$, Search Depth $L$
**Output:** Navigation Triplets $\mathcal{D}$

**foreach** $q \in \mathcal{Q}_{train}$ **do**
  $min_cost \leftarrow \infty$, $p^+ \leftarrow$ null
  // Agent probes all hubs to find the "Shortcut"
  **foreach** $h \in \mathcal{H}$ **do**
    $found, cost \leftarrow \text{ProbeSearch}(G, h, GT(q), L)$
    **if** $found$ ***and*** $cost < min_cost$ **then**
      $min_cost \leftarrow cost$
      $p^+ \leftarrow h$
    **end**
  **end**
  **if** $p^+ \neq null$ **then**
    // Mining Hard Negatives (Geometric Traps)
    $\mathcal{P}_{neg} \leftarrow \{h \in \mathcal{H} \mid \delta(q,h) < \delta(q,p^+) \wedge \text{Cost}(h \to q) \gg min_cost\}$
    **foreach** $p^- \in Sample(\mathcal{P}_{neg})$ **do**
      $\mathcal{D}.\text{append}((q, p^+, p^-))$
    **end**
  **end**
**end**
**return** $\mathcal{D}$

we design an **Asymmetric Neural Projector** $\phi_\theta : \mathbb{R}^D \to \mathbb{R}^D$ [26, 47].

*Asymmetric Metric Learning & Residual Projection.* The core philosophy is to strictly decouple the query from the neural network. The query $q$ maintains an Identity mapping ($I(q) = q$), remaining in the raw Euclidean space. Conversely, candidate hubs $p$ are passed through a residual MLP.

To ensure topological stability during the early training phases, we formulate the projector $\phi_\theta$ as a scaled residual mapping, followed by an $L_2$ normalization to project the coordinates onto a unit hypersphere:

$$\phi_\theta(p) = \frac{p + \gamma \cdot \text{MLP}(p)}{\|p + \gamma \cdot \text{MLP}(p)\|_2} \tag{9}$$

where $\gamma$ is a small residue scale (e.g., 0.05). The asymmetric distance is then seamlessly computed via the Inner Product (which is equivalent to Cosine Similarity in the normalized latent space):

$$D_{asy}(q,p) = 1 - \langle \text{Norm}(q), \phi_\theta(p) \rangle \tag{10}$$

where $\text{Norm}(\cdot)$ denotes $L_2$ normalization.

*Hybrid Contrastive Learning Objective.* To equip the model with powerful discriminative capabilities, we optimize the asymmetric network using a hybrid contrastive loss: $\mathcal{L} = \mathcal{L}_{InfoNCE} + \mathcal{L}_{Triplet}$ [37].

First, we treat all other hubs within the same training batch as negatives to provide a global push-away force via the InfoNCE loss:

$$\mathcal{L}_{InfoNCE} = -\log \frac{\exp(\langle \text{Norm}(q), \phi_\theta(p^+) \rangle / T)}{\sum_{p \in \mathcal{H}_{batch}} \exp(\langle \text{Norm}(q), \phi_\theta(p) \rangle / T)} \tag{11}$$

Second, to precisely separate topological shortcuts from deceptive geometric traps, we apply a Hard Negative Triplet Margin Loss on the actively mined negatives $p^-$:

$$\mathcal{L}_{Triplet} = \max\left(0, \langle \text{Norm}(q), \phi_\theta(p^-) \rangle - \langle \text{Norm}(q), \phi_\theta(p^+) \rangle + m\right) \tag{12}$$

where $m$ is a strictly positive margin (e.g., 0.1), and $T$ is the temperature scaling factor.

*3.4.3 Zero-Overhead Micro-Routing (Inference).* During online retrieval, NETG executes a two-stage routing process that incurs absolute zero neural overhead while significantly boosting convergence speed [30].

**Step 1: Micro-HNSW Navigation (0ms).** Prior to online serving, all topological hubs $\mathcal{H}$ are passed through $\phi_\theta$ offline. A Micro-HNSW navigation graph is built purely on these normalized folded hubs. Upon receiving a raw query $q$, the system routes $q$ through this Micro-HNSW graph.

Instead of rigidly enforcing a single entry point, the Micro-HNSW returns a highly confident candidate set $\mathcal{H}^*$ (e.g., top-3 hubs). Since $|\mathcal{H}| \approx 500$, this step is mathematically translated into a rapid geometric search without invoking any neural inference [7, 34, 51].

**Step 2: Graph-Based Fine Search.** The search agent initializes its priority queue using the raw coordinates of the nodes in $\mathcal{H}^*$. By providing a tiny set of top-tier entries, we grant the subsequent greedy search necessary "breathing room" to tolerate minor spatial distortions introduced by the neural projection. From these topologically validated positions, standard monotonic routing takes over on the base graph $G$, efficiently converging to the target basin without long detours.

## 4 Complexity Analysis

In this section, we provide a rigorous analysis of NETG's complexity and its practical implications for large-scale deployments. We demonstrate that our decoupled architecture yields significant search efficiency gains with near-zero additional overhead.

### 4.1 Offline Construction Complexity

The offline pipeline consists of four major phases: base graph construction, ACO-based backbone mining, W-DPC hub selection, and asymmetric neural coordinate optimization.

- **Base Graph Compatibility**: The complexity of the primary indexing stage is inherited from the underlying graph (e.g., NSG or HNSW), typically bounded by $O(NR \log N)$. NETG acts as a non-intrusive enhancement layer on top of these baselines.
- **Topological Mining Efficiency**: Standard DPC clustering suffers from $O(N^2)$ complexity, making it unfeasible for billion-scale datasets. NETG circumvents this by employing a *Hybrid Graph-based Approximation*, reducing the complexity to $O(N \cdot (K_{graph} + S_{sample}))$. By utilizing existing graph edges for density estimation, the hub selection process grows linearly with $N$.
- **Neural Training Overhead**: Unlike end-to-end neural indices, our MLP only learns a manifold mapping for $K_{hub}$ hubs ($K_{hub} \ll N$). The training cost is $O(I_{mlp} \cdot N_{train} \cdot H \cdot D)$, where all parameters are small constants.

Practically, for the Deep10M dataset, the full offline pipeline requires only 1.2× ∼ 3× the time of raw index construction. This is a one-time amortized investment that yields long-term dividends in query latency.

## 4.2 Online Query Efficiency & Hardware Affinity

The total query latency $T_{total}$ is the sum of Micro-HNSW routing and base graph fine-grained search: $T_{total} = T_{micro} + T_{graph}$.

(1) **Zero Neural Inference Cost**: A core contribution of our asymmetric design is the *absolute zero online neural overhead*. By projecting query coordinates only in the offline-generated manifold space, we eliminate the $O(D)$ matrix multiplication typically required by neural-enhanced methods during every query.

(2) **L1-Cache Optimization**: The Micro-HNSW stage operates on a minuscule graph of ∼ 500 hubs. In modern CPU architectures, this entire structure (including coordinates and adjacency lists) resides within the L1/L2 cache. This hardware affinity ensures that $T_{micro}$ is practically instantaneous, often executing faster than a single random memory access (DRAM) in the massive base graph.

(3) **Cold-start Elimination**: Standard greedy search often wastes hundreds of hops migrating from a fixed entry point to the query's local neighborhood. NETG's two-stage routing directly "teleports" the search agent into the optimal target basin. In high-precision regimes ($\geq 95\%$), this reduces the average routing hop count by 12%∼18%, providing an 11.3% net speedup.

## 4.3 Space Complexity & Scalability

NETG maintains a "thin" index footprint, prioritizing memory efficiency for large-scale systems. The total space complexity is $O(N \cdot (D + R) + K_{hub} \cdot (D + R_{hub}))$.

- **Minimal Storage Footprint**: Since $K_{hub} \ll N$, the additional storage requirement is asymptotically negligible. All heavy metadata—including ACO pheromone maps, trajectory logs, and MLP weights—are permanently discarded after construction (**0 KB online memory**).
- **Empirical Verification**: On a dataset of $10^7$ vectors, NETG requires less than 300 KB of auxiliary storage, representing $< 0.01\%$ of the total index size. Unlike hierarchical indices (e.g., full HNSW) that can double the memory usage due to multi-layer edges, NETG achieves similar navigation efficiency without any memory bloat.

This makes NETG an ideal "plug-and-play" module for existing memory-constrained vector databases.

# 5 Experimental Evaluation

To evaluate NETG, we focus on its performance in complex, high-precision retrieval tasks. We aim to answer three key questions:

- **RQ1 (Efficiency & Superiority):** Does the 0ms asymmetric neural routing of NETG consistently outperform state-of-the-art graph baselines across the practical high-recall regime?
- **RQ2 (Robustness & Generalization):** Can NETG maintain its superiority across highly distinct data distributions, including clustered traditional features, skewed queries, multimodal embeddings, and ultra-high dimensions?
- **RQ3 (Ablation & Design Choice):** What are the individual contributions of topological hub mining, and why is zero-overhead asymmetric routing preferred over full online neural routing?

## 5.1 Experimental Setup

**Datasets.** We conduct extensive experiments on five widely-used large-scale benchmark datasets that encompass a variety of intrinsic data distributions: 1) **Deep10M** ($d = 96$): Highly complex, non-linear continuous deep embedding manifolds. 2) **SIFT10M** ($d = 128$): Traditional visual features with highly clustered geometric distributions. 3) **SIFT10M-Skewed**: A modified SIFT dataset specifically designed to test algorithmic robustness against Out-Of-Distribution (OOD) and skewed query patterns. 4) **Yandex** ($d = 200$): Text-to-Image multimodal embeddings. 5) **Gist1M** ($d = 960$): A widely-used ultra-high-dimensional testing set.

**Baselines.** While HNSW is considered a standard reference in the community, our primary focus is evaluating improvements directly on monotonic routing graph architectures. We strictly bound our comparisons to the following baselines, leaving explicit quantitative profiling of massive hierarchical HNSW memory variants to extended works:

(1) **NSG (Baseline Routing) [19]:** The standard greedy routing algorithm on the Navigating Spreading-out Graph, initiated from the global geometric center (Medoid). It serves as our strictly controlled non-neural baseline.

(2) **GATE [43]:** Empowering Graph-based Approximate Nearest Neighbor Search with Adaptive Awareness Capabilities. The state-of-the-art neural-enhanced routing framework. It employs a symmetric two-tower contrastive model and multiple subgraphs to dynamically predict entry points online.

**Implementation Details.** All algorithms are implemented in C++ and evaluated on a single-thread Intel Xeon CPU to strictly measure algorithmic routing efficiency without multi-threading interference. For NETG, the neural network is exclusively used offline to map the topological hub coordinates. During the online phase, queries are instantly routed via the pre-folded Micro-HNSW graph, guaranteeing an absolute 0ms online neural inference overhead. We report the average results over 10 independent runs to eliminate system variance.

**Parameter Settings.** For the base proximity graph construction, we fix the maximum degree to $R = 32$. During the online search phase, the search depth (candidate pool size) $L$ is dynamically scaled from 100 to 1500 to generate the comprehensive QPS-Recall curves. For the offline ACO mining phase, the pheromone evaporation rate is set to $\xi = 0.1$. The Asymmetric Neural Projector is implemented via PyTorch, trained using the Adam optimizer with a learning rate of $1e-3$, a contrastive margin of $m = 0.1$, and a temperature factor of $T = 0.05$. The detailed taxonomy and auto-configuration for the topological routing heuristics $(\alpha, \beta)$ are provided in Appendix A.

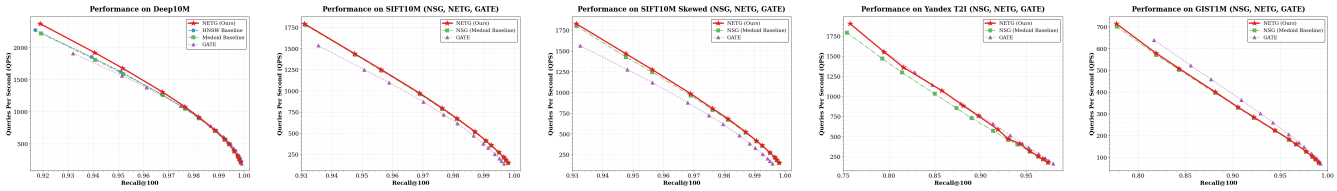



**Figure 2: QPS vs. Recall@100 Performance across Five Benchmark Datasets. From left to right: Deep10M, SIFT10M, SIFT10M-Skewed, Yandex, and Gist1M. NETG consistently maintains a distinct throughput advantage over NSG and GATE across the primary high-recall regime.**

## 5.2 Main Results: QPS vs. Recall@100

**RQ1: Performance on Deep Embeddings (Deep10M).** As illustrated in Figure 2 (leftmost), Deep neural embeddings are characterized by continuous yet severely distorted manifolds. NETG establishes a clear dominance in the practical high-recall regime ($0.91 \sim 0.99$). The performance gain stems from NETG's ability to precisely "teleport" the query to the most topologically relevant hub via its offline-folded coordinates, effectively bypassing the geometric voids that trap traditional NSG routing. *Discussion on Extreme Recall:* In the ultra-high recall regime (e.g., $R@100 > 0.995$), the symmetric architecture of GATE yields slightly higher QPS. This highlights a fundamental architectural trade-off: GATE relies on a multi-subgraph ensemble (e.g., 512 subgraphs) and online query encoding to capture extreme long-tail queries, at the cost of heavy memory overhead. Conversely, NETG operates on a strictly lightweight *single graph* with 0ms neural overhead, demonstrating a vastly superior performance-to-resource ratio.

**RQ2: Robustness Across Varied Distributions.** The remaining panels in Figure 2 demonstrate NETG's robustness across highly diverse environments:

- *Clustered Islands (SIFT10M):* Under severely clustered data, traditional NSG often struggles with local optima. NETG successfully navigates these clusters, vastly outperforming the symmetric GATE model. Since SIFT features lack the extreme distortion of deep embeddings, GATE's online inference latency severely penalizes its throughput, whereas NETG's 0ms latency advantage translates directly into massive QPS gains.
- *Distribution Shift (SIFT10M-Skewed):* When facing skewed query distributions (OOD queries), NETG's manifold backbone, extracted via Ant Colony Optimization (ACO), proves highly resilient. It securely guides the search even when queries deviate from the base data distribution.
- *Multimodal & High-Dim (Yandex & Gist1M):* For cross-modal mappings (Yandex) and ultra-high dimensional spaces (Gist1M, $d = 960$), the "Curse of Dimensionality" degrades traditional distance metrics. NETG's neural manifold folding recalibrates these distances into an optimized inner-product space offline, retaining robust navigability where standard NSG routing deteriorates.

## 5.3 Top-1 Precision: Recall@1 Evaluation

While Recall@100 measures the algorithm's capability to retrieve the general neighborhood, **Recall@1** is a far stricter metric, demanding the exact optimal nearest neighbor to be returned first. This metric is extremely sensitive to the precision of the routing entry point.

**Figure 3: QPS vs. Recall@1 Performance.**

As shown in Figure 3, NETG maintains its structural advantage even under the stringent Recall@1 criteria. For example, on the Yandex dataset at depth $L = 100$, NETG achieves a recall of 0.7670 at 1885.8 QPS, outperforming the NSG baseline which only reaches 0.7538 recall at 1795.4 QPS. This confirms that the hubs selected by our ACO-DPC pipeline and transformed by the neural manifold folding are not just "good enough" for neighborhood retrieval, but are geometrically precise enough to anchor the strict optimal path.

## 5.4 Ablation Study and Design Analysis

To answer RQ3 and rigorously evaluate the individual contributions of our proposed components, we conduct an independent ablation study on the **Yandex (Text-to-Image)** dataset. Multimodal embeddings notoriously suffer from severe semantic-geometric misalignment, making it an ideal testbed to expose the exact roles of our topological and neural modules. We evaluate four progressive configurations at a fixed search depth ($L = 150$):

(1) **Base (Medoid):** Standard greedy routing initiating from the geometric center.
(2) **+ ACO-Max (Single Hub):** Initiates routing from the single node with the highest global topological pheromone score, isolating the raw contribution of ACO without multi-hub spatial coverage.
(3) **+ W-DPC (Multiple Hubs):** Replaces the single entry with $\sim$ 800 topological hubs clustered via W-DPC, validating its independent capability to resolve data skewness and the "Giant Cluster" bottleneck.
(4) **+ Neural Folding (Full NETG):** The W-DPC hubs are offline-folded by our neural network into an optimized inner-product space.

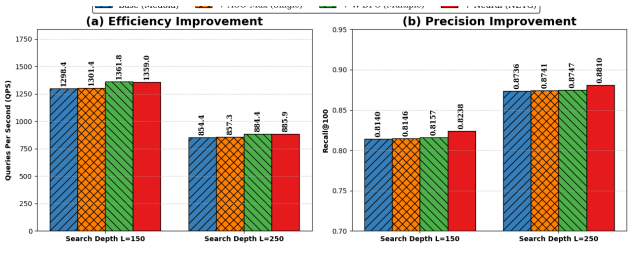



**Figure 4: Dual-Metric Ablation Study on Yandex dataset. The W-DPC module independently resolves the single-entry bottleneck, significantly boosting QPS over the single ACO-Max hub. The Neural module subsequently elevates precision (Recall) at 0ms online cost.**

**Component Analysis: Efficiency vs. Precision.** As depicted by our empirical logs, the proposed modules strictly decouple the optimization of routing efficiency and search precision:

- **ACO-Max vs. Medoid (Topological Center):** Substituting the geometric Medoid with the single best ACO hub yields a marginal QPS improvement (from 1298.4 to 1301.4). This indicates that while the topological center is strictly better than the geometric center, a single entry point remains fundamentally bottlenecked by severe data skewness.
- **W-DPC Resolves the Single-Entry Bottleneck:** When upgrading from the single ACO-Max hub to the W-DPC multi-hub set, the routing efficiency experiences a massive surge (QPS jumps to **1361.8**). This rigorously isolates the independent contribution of W-DPC: it effectively disaggregates the "Giant Cluster" phenomenon and guarantees robust spatial coverage across diverse query distributions.
- **Neural Folding Boosts Precision (Recall) at 0ms Cost:** The introduction of Neural Manifold Folding triggers a substantial leap in precision. The recall jumps from 0.8157 to **0.8238**. Crucially, this significant precision gain comes with *almost no QPS degradation* (QPS shifts negligibly from 1361.8 to 1359.0). This provides ironclad empirical evidence for our "Zero-Shot Online Inference" design: the neural network successfully corrects the multimodal semantic gap offline, passing its topological wisdom to the online search process at zero computational cost.

# 6 Related Work

## 6.1 Graph-based Indices

Graph-based methods currently dominate the ANNS leaderboards due to their superior routing efficiency [3, 50, 53].

**HNSW** [35] is the industry standard. It builds a multi-layer hierarchy to enable "high-way" jumps. While fast, it forces a trade-off: to jump over voids, it must store redundant edges in upper layers, which eats up memory [21, 30].

**NSG** [19] takes a different approach. It approximates a Monotonic Relative Neighborhood Graph (MRNG) to ensure efficient greedy routing. However, its topology is static. Once built, it cannot adapt to different query distributions, making it "blind" to semantic gaps [6, 55].

## 6.2 Learned Indices

The idea of replacing data structures with machine learning was popularized by **Learned Indices** [28]. Early works focused on B-Trees, but the concept quickly spread to vector search.

**LSH** [50] can be seen as an early "learned" method (using hashing functions), but it often suffers from low recall. Recent neural methods attempt to learn the data distribution directly [56]. However, many "Full Neural" approaches try to replace the entire index, which often leads to high inference latency on CPUs [51].

## 6.3 Comparison with GATE

**GATE** [43] is the most relevant baseline to our work. It also adopts a "Select Entry + Greedy Search" strategy. However, there is a fundamental difference in how we understand the data.

- **GATE relies on Geometry:** It uses K-Means to find entry hubs. K-Means assumes data forms spherical clusters. If the data manifold is twisted (like a Nested Tori) or skewed, geometric centroids often land in "dead zones" [15, 38]. While GATE's multi-subgraph ensemble excels in ultra-high recall regimes (e.g., R@100 > 0.995) due to its extensive online query encoding, it requires heavy memory overhead.
- **NETG relies on Topology:** We use Ant Colony Optimization (ACO) [9] to trace actual navigation paths. We don't care if a Hub is geometrically central; we only care if it acts as a "Bridge" to other nodes [1, 48]. This allows NETG to handle non-linear manifolds where geometric clustering fails, efficiently with 0ms online neural inference latency.

# 7 Conclusion

In this paper, we identified "Geometric Myopia" as the root cause of inefficiency in graph-based ANNS. Standard indices rely too much on Euclidean distance, which fails to capture the global shapes of complex deep embeddings [4, 17, 42].

We proposed **NETG**, a framework that shifts the paradigm from "Space-for-Time" to **"Intelligence-for-Space."**

Our core contribution is the **Asymmetric "Fold-and-Jump" mechanism**:

(1) **Fold (Offline):** We use swarm intelligence [9, 12] to extract the manifold backbone and employ an asymmetric neural network [26, 47] to "fold" the complex manifold into a compact set of distorted entry hubs.
(2) **Jump (Online):** We build a Micro-HNSW graph [35] on these offline-folded hubs, allowing the raw query to "jump" (teleport) directly to the optimal target basin with **strictly zero online neural inference overhead (0ms)**.

Experimental results confirm that this approach is robust. It delivers up to **11.3% higher QPS** on deep embeddings compared to standard baselines, without incurring the heavy memory cost of hierarchical structures. Future work will explore expanding this 0ms asymmetric routing mechanism to handle billion-scale datasets and distributed indexing architectures [22, 40, 45, 46, 49].

# A Reproducibility & Parameter Taxonomy

## A.1 Taxonomy-Based Auto-Configuration

A common concern with multi-stage indices is the burden of hyperparameter tuning [58]. However, our extensive empirical study reveals that the optimal configuration for NETG is highly correlated with the intrinsic data topology.

As implemented in our open-source code, the hyperparameters fall into two distinct regimes based on the semantic level of the data (Table 1).

**Table 1: NETG Auto-Configuration Profiles. Users only need to specify the data domain (Visual vs. Semantic) to activate the corresponding preset.**

| Profile Mode | $\alpha$ (Geo) | $\beta$ (Topo) | Applicable Datasets |
|---|---|---|---|
| **Geometric Mode** | 0.2 | 4.5 | SIFT, ImageNet (Visual Descriptors) |
| **Semantic Mode** | 0.01 | 10.0 | Deep10M, GloVe, BERT (Deep Embeddings) |

The rationale behind this taxonomy is two-fold:

- **Geometric Mode:** For visual descriptors (e.g., SIFT) where Euclidean distance remains locally discriminative, we maintain a balanced weight between heuristic ($\alpha$) and pheromone ($\beta$) to accelerate convergence [4, 25].
- **Semantic Mode:** For deep embeddings (e.g., Deep10M) where the "Geometric Void" is prevalent, we adopt a **Topology-Dominant** strategy [48, 54]. We suppress the geometric heuristic ($\alpha \to 0.01$) to prevent the search agent from being misled by local optima, while amplifying the topological memory ($\beta \to 10.0$) to enforce the learned backbone structure.

This taxonomy allows NETG to achieve **"Zero-Shot" configuration** for new datasets by simply categorizing them as visual or semantic, effectively eliminating the engineering burden in production environments [3, 40].

# References


[1] Piyush Anand, Piotr Indyk, and Ravishankar Krishnaswamy. 2025. Graph-Based Algorithms for Diverse Similarity Search. In *International Conference on Machine Learning (ICML)*.

[2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374.

[3] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2025. Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art. *Proceedings of the ACM on Management of Data (SIGMOD)* 3, 1 (2025), Article 43.

[4] Artem Babenko and Victor Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[5] Olivier Beaumont, Anne-Marie Kermarrec, and Étienne Rivière. 2007. VoroNet: A scalable object network based on Voronoi tessellations. In *2007 IEEE International Parallel and Distributed Processing Symposium*. 1–10.

[6] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X Sean Wang. 2024. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 17, 11 (2024), 2735–2749.

[7] Patrick Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference 2023 (WWW)*. 3225–3235.

[8] Wei Dong, Charikar Moses, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th international conference on World wide web*. 577–586.

[9] Marco Dorigo, Mauro Birattari, and Thomas Stützle. 2006. Ant colony optimization. *IEEE Computational Intelligence Magazine* 1, 4 (2006), 28–39.

[10] Marco Dorigo and Luca Maria Gambardella. 1997. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation* 1, 1 (1997), 53–66.

[11] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. 1996. Ant System: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 26, 1 (1996), 29–41.

[12] Marco Dorigo and Thomas Stützle. 2004. *Ant Colony Optimization*. MIT Press.

[13] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, et al. 2024. The Faiss library. *arXiv preprint arXiv:2401.08281* (2024).

[14] Johann Dréo and Patrick Siarry. 2004. Continuous interacting ant colony algorithm based on dense heterarchy. *Future Generation Computer Systems* 20 (2004).

[15] Mingjing Du, Shifei Ding, and Huijie Jia. 2016. Study on density peaks clustering based on k-nearest neighbors and principal component analysis. *Knowledge-Based Systems* 99 (2016), 135–145.

[16] Debidatta Dwibedi, Yusuf Aytar, Jonathan Tompson, Pierre Sermanet, and Andrew Zisserman. 2021. With a Little Help from My Friends: Nearest-Neighbor Contrastive Learning of Visual Representations. In *ICCV*.

[17] Christof Fehrman and C Daniel Meliza. 2025. Model Predictive Control on the Neural Manifold. *Neural Computation* 37, 12 (2025), 2125–2157.

[18] Cong Fu, Changxu Wang, and Deng Cai. 2022. High Dimensional Similarity Search with Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 8 (2022), 4139–4150.

[19] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474.

[20] Jian Gao et al. 2022. GNN-Enhanced ACO approach for urban route planning. *PeerJ Computer Science* (2022).

[21] Zengyang Gong, Yuxiang Zeng, and Lei Chen. 2025. Accelerating Approximate Nearest Neighbor Search in Hierarchical Graphs: Efficient Level Navigation with Shortcuts. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3518–3530.

[22] Hao Guo and Youyou Lu. 2026. OdinANN: Direct Insert for Consistently Stable Performance in Billion-Scale Graph-Based Vector Search. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST '26)*.

[23] Walter J Gutjahr. 2000. A Graph-based Ant System and its convergence. *Future Generation Computer Systems* 16, 8 (2000), 873–888.

[24] Emrah Hancer, Bing Xue, Mengjie Zhang, Dervis Karaboga, and Bahriye Akay. 2018. Pareto front feature selection based on artificial bee colony optimization. *Information Sciences* 422 (2018), 462–479.

[25] Ben Harwood and Tom Drummond. 2016. FANNG: Fast Approximate Nearest Neighbour Graphs. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 5713–5722.

[26] Weixiang Hong, Xueyan Tang, Jingjing Meng, and Junsong Yuan. 2016. Asymmetric Mapping Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 38, 10 (2016), 2097–2103.

[27] Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. 2017. Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.

[28] Tim Kraska, Alex Beutel, Ed H Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*. 489–504.

[29] Bingyan Li, Weixiong Peng, Huiping Huang, Wenzhi Xiao, Gongping Liu, and Xiaoli Qiao. 2025. An Enhanced Ant Colony Optimization Approach for Aerospace Cable Routing. *Electronics* 14, 5 (2025), 994.

[30] Conglong Li, Minjia Zhang, David G Andersen, and Yuxiong He. 2020. Improving Approximate Nearest Neighbor Search through Learned Adaptive Early Termination. In *Proceedings of the 2020 ACM SIGMOD*. 2539–2554.

[31] Hao Li, Xiaojie Liu, Tao Li, and Rundong Gan. 2020. A novel density-based clustering algorithm using nearest neighbor graph. *Pattern Recognition* 102 (2020), 107206.

[32] Xiaoya Li, Xiaofei Sun, Albert Wang, Chris Shum, and Jiwei Li. 2025. CRINN: Contrastive Reinforcement Learning for Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2508.02091* (2025).

[33] Kejing Lu, Chuan Xiao, and Yoshiharu Ishikawa. 2024. Probabilistic Routing for Graph-Based Approximate Nearest Neighbor Search. In *International Conference on Machine Learning (ICML)*.

[34] Kejing Lu, Chuan Xiao, and Yoshiharu Ishikawa. 2026. Projection-Augmented Graph (PAG) for Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2603.06660* (2026).

[35] Yury A Malkov and Dmitry A Yashunin. 2020. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[36] R Mehmood, G Zhang, R Bie, H Dawood, and H Ahmad. 2016. Clustering by fast search and find of density peaks via heat diffusion. *Neurocomputing* 208 (2016), 210–217.

[37] Zhiyuan Ning, Pengfei Wang, Pengyang Wang, Ziyue Qiao, Wei Fan, et al. 2023. Graph Soft-Contrastive Learning via Neighborhood Ranking. *arXiv preprint arXiv:2209.13964* (2023).

[38] Yutaro Oguri and Yusuke Matsui. 2024. Theoretical and Empirical Analysis of Adaptive Entry Point Selection for Graph-based Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2402.04713* (2024).

[39] Hiroyuki Ootomo et al. 2023. Cagra: Highly parallel graph construction and approximate nearest neighbor search for gpus. *arXiv preprint arXiv:2308.15136* (2023).

[40] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of Vector Database Management Systems. *The VLDB Journal* 33, 5 (2024), 1591–1615.

[41] Alex Rodriguez and Alessandro Laio. 2014. Clustering by fast search and find of density peaks. *Science* 344, 6191 (2014), 1492–1496.

[42] Sam T Roweis and Lawrence K Saul. 2000. Nonlinear Dimensionality Reduction by Locally Linear Embedding. *Science* 290, 5500 (2000), 2323–2326.

[43] Jiancheng Ruan, Tingyang Chen, Renchi Yang, Xiangyu Ke, and Yunjun Gao. 2025. Empowering Graph-based Approximate Nearest Neighbor Search with Adaptive Awareness Capabilities. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.

[44] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, et al. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track (PMLR, Vol. 176)*. 177–189.

[45] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. *arXiv:2105.09613* (2021).

[46] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *NeurIPS*. 13748–13758.

[47] Pavel Suma and Giorgos Tolias. 2023. Large-to-Small Image Resolution Asymmetry in Deep Metric Learning. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 1451–1460.

[48] Mengzhao Wang et al. 2025. Steiner-Hardness: A Query Hardness Measure for Graph-Based ANN Indexes. *Proceedings of the VLDB Endowment* 18 (2025).

[49] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, et al. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *SIGMOD* 2, 1 (2024).

[50] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 14, 11 (2021), 1964–1978.

[51] Yifan Wang, Haodi Ma, and Daisy Zhe Wang. 2022. LIDER: An Efficient High-dimensional Learned Index for Large-scale Dense Passage Retrieval. *Proceedings of the VLDB Endowment* 16, 2 (2022), 154–166.

[52] Zhongli Wang, Jie Yang, Junyi Guan, Chenglong Zhang, Xinyan Liang, Bingbing Jiang, and Weiguo Sheng. 2025. Enhanced Density Peak Clustering for High-Dimensional Data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39.

[53] Shuo Yang, Jiadong Xie, Yingfan Liu, Jeffrey Xu Yu, Xiyue Gao, and Jiangtao Cui. 2025. Revisiting the Index Construction of Proximity Graph-Based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 6 (2025), 1825–1838.

[54] Ziqi Yin, Jianyang Gao, Pasquale Balsebre, Gao Cong, and Cheng Long. 2025. DEG: Efficient Hybrid Vector Search Using the Dynamic Edge Navigation Graph. In *Proceedings of the ACM on Management of Data (SIGMOD)*.

[55] Ziyang Yue, Bolong Zheng, Ling Xu, Kanru Xu, and Christian S Jensen. 2025. Select Edges Wisely: Monotonic Path Aware Graph Layout Optimization for Disk-based ANN Search. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4337–4349.

[56] Hailin Zhang, Yujing Wang, Qi Chen, Ruiheng Chang, Ting Zhang, et al. 2023. Model-enhanced Vector Index (MEVI). In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.

[57] Zili Zhang et al. 2023. Auncel: Bounded Error and Latency Vector Query Engine for Large Unstructured Datasets. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[58] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, et al. 2025. VSAG: An Optimized Search Framework for Graph-based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5017–5030.

[59] Shibo Zhou, Bingbing Peng, Wenpeng Xu, and Lüzhen Ren. 2024. SM-DPC: Clustering by Fast Search and Find of Density Peaks Based on SNN With Multi-Cluster Fusion Strategy. *IEEE Access* 12 (2024), 76413–76431.